

Compositional Energy-Based Reasoning for Symbolic Algebra

Matt Krasnow
Harvard College
mkrasnow@college.harvard.edu

Jaray Liu
Harvard College
jarayliu@college.harvard.edu

December 12, 2025

Abstract

We introduce a compositional approach to symbolic algebra reasoning with energy-based models (EBMs). Instead of training a single model on complete multi-step solution sequences, we train separate energy functions for individual algebraic rules and compose them at inference time to solve multi-rule problems. Building on IRED (Iterative Reasoning through Energy Diffusion), we define rule-level energies and sum them to guide optimization toward valid solution states. On multi-rule problems, our compositional model improves accuracy by 6.2 percentage points over a monolithic IRED baseline (13.2% vs 7.1%), an 87% relative gain. The advantage grows with problem length, from a modest +0.7% on 2-rule problems to +16.0% on 4-rule chains, where performance more than doubles (28.0% vs 12.0%). The modular design supports runtime constraint injection and provides a practical mechanism for systematic generalization in neural reasoning systems.

1 Introduction

1.1 Problem Statement and Motivation

Neural approaches to symbolic reasoning often learn from complete multi-step trajectories, which ties generalization to the training distribution of rule sequences. This setup makes it difficult to solve new combinations of known steps without retraining. We study compositional generalization in symbolic algebra: can a model solve multi-rule problems after training only on single-rule data?

1.2 Contributions

We make three contributions:

- We define and evaluate rule-level energy composition for symbolic algebra reasoning.
- We extend IRED with modular, rule-specific energy functions composed by energy summation at inference time.
- We show improved performance on multi-step algebraic problems while matching single-rule performance.

2 Related Work

2.1 Energy-Based Reasoning

Energy-based models (EBMs) assign a scalar energy to each configuration and perform inference by searching for low-energy states [LeCun et al., 2006]. This view connects discriminative prediction and structured inference by casting prediction as optimization over an energy landscape.

IREN (Iterative Reasoning through Energy Diffusion) [Du et al., 2024] formulates reasoning as optimization over annealed energy landscapes. The model learns an energy that scores compatibility between inputs and outputs, then applies gradient-based diffusion steps at inference time. IREN reports strong out-of-distribution performance on structured tasks, including Sudoku, path-finding, and matrix completion, by adapting the number of optimization steps at inference time [Du et al., 2024].

Our work keeps IREN’s optimization-based inference but changes the modeling granularity. IREN typically uses a monolithic energy function per task family, which entangles multiple constraints and rules. We instead train separate energies for individual algebraic rules (distribution, combining like terms, isolation, division) and compose them at inference. This design matches classical modular constraint systems, but we implement it within a differentiable, energy-based pipeline.

Other work explores compositional structure in constraint solving with diffusion-style methods. Diffusion-CCSP [Yang et al., 2023b] factors continuous constraint satisfaction problems into simpler constraints and composes them at inference. Recurrent Transformer approaches to constraint satisfaction [Yang et al., 2023a] solve CSP instances end-to-end without explicit energies, emphasizing learned constraint satisfaction rather than explicit compositional energy terms. These methods focus on continuous or general CSP settings, while we target discrete symbolic algebra with explicit rule semantics.

In mathematical and symbolic reasoning, energy-based methods remain less common than sequence models. Large-scale studies of neural mathematical reasoning [Saxton et al., 2019] and symbolic mathematics [Lample and Charton, 2019] primarily use sequence-to-sequence or Transformer architectures. These methods often learn monolithic mappings from problems to solutions rather than reusable, rule-specific components. We address this gap by testing whether composing interpretable rule-level energies improves multi-step generalization.

2.2 Compositional Reasoning and Neuro-Symbolic Methods

Compositional reasoning aims to solve new problems by recombining learned primitives. Neuro-symbolic approaches often encode structure explicitly while using gradient-based learning. Neural Logic Machines (NLMs) [Dong et al., 2019] build hierarchies of neural modules that implement lifted logical operations over relational data and can generalize from small to larger instances. Differentiable inductive logic programming methods [Evans and Grefenstette, 2018] learn Horn-clause-like rules from data, combining inductive biases from ILP with neural robustness. Neural theorem proving [Minervini et al., 2018] relaxes proof search into differentiable operations over embeddings.

Our method sits between explicit symbolic search and end-to-end sequence modeling. Like neuro-symbolic systems, we align model structure with human-interpretable rules. Like IREN, we perform inference through continuous optimization rather than discrete proof search or program execution. We focus on rule-level energy composition and quantify its effect on multi-step algebraic generalization.

2.3 Symbolic AI, Systematic Generalization, and Algebraic Reasoning

Computer algebra systems achieve compositional generalization by encoding algebraic rules explicitly and applying deterministic rewriting procedures. These systems excel at symbolic manipulation but rely on hand-engineered rule sets and do not learn from data.

A central question in machine learning asks whether neural networks can acquire systematic generalization from data. Lake and Baroni [Lake and Baroni, 2018] introduced SCAN to test compositional generalization from limited combinations of primitives. Standard sequence models often fail to extrapolate to new combinations even when they perform well on i.i.d. splits. Saxton et al. [Saxton et al., 2019] constructed synthetic math suites, including algebra, to probe generalization across curricular splits and found similar gaps between i.i.d. performance and compositional generalization.

In symbolic mathematics, Lample and Charton [Lample and Charton, 2019] showed that sequence models trained on synthetic data can solve tasks such as integration and differential equation solving. These models learn holistic mappings, which makes it difficult to attribute generalization to explicit rule reuse rather than distributional pattern matching.

We target this gap with a controlled setup: single-variable linear equations with a fixed set of algebraic rules and symbolic verification via SymPy. We train only on single-rule data and test on multi-rule problems, which mirrors compositional splits in the SCAN tradition [Lake and Baroni, 2018]. Our approach ties generalization to the composition of interpretable rule-level energies rather than to a single monolithic mapping.

3 Method

3.1 Problem Formulation

We solve single-variable linear equations of the form $ax + b = cx + d$ using sequences of rule applications. Let $\mathcal{R} = \{r_1, r_2, r_3, r_4\}$ denote four algebraic rules:

- r_1 : **Distribution** — $a(bx + c) \rightarrow abx + ac$
- r_2 : **Combining like terms** — $ax + bx \rightarrow (a + b)x$
- r_3 : **Isolation** — $ax + b = c \rightarrow ax = c - b$
- r_4 : **Division** — $ax = b \rightarrow x = \frac{b}{a}$ (when $a \neq 0$)

Given an initial equation e_0 , the goal is to transform it into a solved form $x = v$ for some $v \in \mathbb{R}$. We define the state space $\mathcal{X}$ as the set of valid expressions representable in our domain. Each rule r_i defines a transformation function $f_{r_i} : \mathcal{X} \rightarrow \mathcal{X}$.

We represent a solution process as a length- T sequence of states $\{e_t\}_{t=0}^T$ where $e_{t+1} = f_{r_{i_{t+1}}}(e_t)$ and the terminal state e_T has the form $x = v$.

3.2 Rule-Level Energy Functions

We train separate energy functions $E_i : \mathcal{X} \rightarrow \mathbb{R}$ for each rule $r_i \in \mathcal{R}$. Each E_i assigns low energy to expressions aligned with the effect of rule r_i and higher energy to mismatched expressions.

For rule r_i , we construct a dataset $\mathcal{D}_i = \{(e^{(j)}, s^{(j)})\}_{j=1}^{N_i}$ where each example contains an initial expression $e^{(j)}$ and a target solution $s^{(j)}$ obtained by applying only rule r_i . We parameterize E_i by a neural network and train it with an IRED-style objective:

$$\mathcal{L}_i = \mathbb{E}_{(e,s) \sim \mathcal{D}_i} [E_i(s) - E_i(\tilde{s}) + \alpha \|\nabla_x E_i(x)\|^2], \quad (1)$$

where $\tilde{s}$ is an incorrect (negative) solution and the gradient penalty encourages smoother energy landscapes for optimization-based inference.

3.3 Energy Composition for Multi-Rule Problems

For multi-rule problems, we compose rule energies by weighted summation. Let $\mathcal{R}_{\text{active}} \subseteq \mathcal{R}$ denote rules relevant to a given problem. We define the compositional energy:

$$E_{\text{comp}}(x; \mathcal{R}_{\text{active}}) = \sum_{r_i \in \mathcal{R}_{\text{active}}} \lambda_i E_i(x), \quad (2)$$

where $\lambda_i \geq 0$ are composition weights. We use uniform weights $\lambda_i = 1$ in our experiments.

Each E_i encodes a rule-specific preference. Summation combines these preferences into a single landscape that encourages states compatible with multiple rules, enabling optimization to move through multi-step solution trajectories.

3.4 Inference Algorithm

At inference, we adapt IRED’s optimization procedure to use E_{comp} . Given an initial equation e_0 and active rules $\mathcal{R}_{\text{active}}$, we optimize in the energy landscape defined by E_{comp} .

Algorithm 1 Compositional Energy-Based Algebraic Reasoning

- 1: **Input:** Initial equation e_0 , active rules $\mathcal{R}_{\text{active}}$, steps T
 - 2: **Output:** Solution expression
 - 3: Initialize $x^{(0)} \leftarrow \text{encode}(e_0)$
 - 4: Define $E_{\text{comp}}(x) \leftarrow \sum_{r_i \in \mathcal{R}_{\text{active}}} E_i(x)$
 - 5: **for** $t = 0$ to $T - 1$ **do**
 - 6: $g^{(t)} \leftarrow -\nabla_x E_{\text{comp}}(x^{(t)})$ {Energy gradient}
 - 7: $x^{(t+1)} \leftarrow x^{(t)} + \eta \cdot g^{(t)} + \sqrt{2\eta\beta^{-1}}\epsilon^{(t)}$ {Langevin dynamics}
 - 8: Apply annealing: $\beta^{(t+1)} \leftarrow \beta^{(t)} \cdot \gamma$
 - 9: **end for**
 - 10: **return** $\text{decode}(x^{(T)})$ {Convert to algebraic expression}
-

The procedure alternates energy-reducing gradient steps with stochastic perturbations to escape poor local minima. The annealing schedule reduces noise over time to promote convergence.

3.5 Zero-Shot Multi-Rule Generalization

The rule-level decomposition supports zero-shot generalization to multi-rule problems. During training, each E_i learns a rule-specific energy landscape from single-rule data. At inference, E_{comp} combines these learned preferences into a joint landscape that guides optimization through multi-step solutions without training on multi-rule sequences.

4 Experimental Setup

4.1 Dataset Construction

We build datasets for single-rule training and multi-rule evaluation. The generator targets broad coverage of expression structure while controlling difficulty and ensuring symbolic verifiability.

4.1.1 Single-Rule Training Datasets

For each rule $r_i \in \mathcal{R}$, we generate a dataset $\mathcal{D}_i$ containing problems solvable by only that rule:

- **Distribution ($\mathcal{D}_1$):** Problems of the form $a(bx + c) = d$, where a, b, c, d are sampled integers from $[-10, 10] \setminus \{0\}$. Examples: $3(2x + 5) = 21$, $-2(x - 4) = 6$.
- **Combining like terms ($\mathcal{D}_2$):** Problems of the form $ax + bx + c = d$, with coefficients sampled from the same range. Examples: $3x + 2x - 4 = 11$, $x - 3x + 7 = -5$.
- **Isolation ($\mathcal{D}_3$):** Problems of the form $ax + b = c$ where isolation yields a division-ready equation. Examples: $2x + 3 = 7$, $-x - 5 = 2$.
- **Division ($\mathcal{D}_4$):** Problems already in the form $ax = b$. Examples: $4x = 12$, $-3x = 9$.

Each dataset includes the initial equation and the ground-truth solution $x = v$ computed with SymPy. We enforce unique rational solutions to avoid ambiguous targets.

4.1.2 Multi-Rule Test Set

Our evaluation set contains 175 problems that require 2 to 4 sequential rule applications:

- **2-rule problems (75 instances):** Two-rule combinations, such as distribution followed by isolation or combining like terms followed by division.
- **3-rule problems (60 instances):** Three-step chains such as distribution $\rightarrow$ combining like terms $\rightarrow$ isolation.
- **4-rule problems (40 instances):** Problems that require all four rules.

We generate multi-rule problems by starting from a solved form $x = v$ and applying inverse transformations to produce a complex initial equation. This construction guarantees unique, verifiable solutions.

Example multi-step problem:

$$\text{Initial: } 2(3x + 4) + 5x = 19 \tag{3}$$

$$\text{Distribution: } 6x + 8 + 5x = 19 \tag{4}$$

$$\text{Combining: } 11x + 8 = 19 \tag{5}$$

$$\text{Isolation: } 11x = 11 \tag{6}$$

$$\text{Division: } x = 1 \tag{7}$$

4.2 Implementation Details

4.2.1 Expression Encoding

We represent expressions as fixed-size vectors using a custom tokenization scheme. We parse each expression into an abstract syntax tree (AST) and convert it into tokens for operations, variables, and coefficients. Learnable token embeddings and positional encodings for tree structure produce 128-dimensional representations.

4.2.2 Network Architecture

Each rule-specific energy function E_i uses a multi-layer perceptron (MLP):

- Input: 128 dimensions (expression encoding)
- Hidden layers: 3 layers with 256 units each
- Activation: ReLU with dropout (rate = 0.1)
- Output: scalar energy value

All rule networks share this architecture but use independent parameters, with about 200K parameters per rule.

4.2.3 Training Protocol

We train each E_i independently on $\mathcal{D}_i$ using an IRED-style training loop:

- Optimizer: Adam with learning rate 10^{-3}
- Batch size: 32
- Training epochs: 100 per rule
- Negative sampling: random incorrect solutions for contrastive learning
- Regularization: $\alpha = 0.01$ for the gradient penalty in Equation (1)

4.3 Baselines

We compare against:

- **Monolithic IRED:** One energy function trained on complete multi-step sequences. It uses the same architecture as the rule models but trains on full problem-solution pairs.
- **Sequence-to-Sequence Transformer:** An encoder-decoder Transformer trained to map problem statements to solutions as text. It uses 6 layers, 8 attention heads, and 512 hidden dimensions.
- **Random Composition:** Our compositional method with randomly initialized energies to test whether composition alone drives gains.
- **Oracle Composition:** A reference upper bound using hand-crafted energies that recognize when each rule should apply.

4.4 Evaluation Metrics

We evaluate with:

- **Symbolic Correctness:** Exact-match accuracy using SymPy to verify equivalence between predicted and ground-truth solutions.
- **L2 Solution Distance:** Euclidean distance between predicted and correct numeric solutions for incorrect outputs.

- **Syntactic Validity:** Fraction of outputs that parse as valid algebraic expressions.
- **Intermediate Step Analysis:** For n -rule problems, accuracy on prefixes that require only the first $k < n$ rules.

4.5 Experimental Protocol

We train rule-specific models on single-rule datasets and evaluate on both single-rule and multi-rule sets. We select hyperparameters by grid search on a small validation set (10% of training data) and report held-out test performance.

We evaluate both *in-domain generalization* (multi-rule problems with familiar coefficient ranges) and *out-of-domain generalization* (coefficients from $[-50, 50]$, unseen during training) to assess robustness.

5 Results

We report performance on single-rule and multi-rule algebraic problems. Compositional energy functions improve multi-step reasoning while maintaining comparable single-rule performance.

5.1 Main Performance Comparison

Table 1 summarizes the core results. Monolithic IRED and the compositional model match on single-rule problems (about 29.6% accuracy). On multi-rule problems, the compositional model improves accuracy by 6.2 percentage points (13.2% vs 7.1%), an 87% relative improvement over the monolithic baseline.

Table 1: Performance comparison on algebraic equation solving.

Model	Single-Rule Acc	Multi-Rule Acc	
Monolithic IRED	29.6%	7.1%	
Compositional (Ours)	29.6%*	13.2%	*Compositional uses rule-specific
<i>Improvement</i>	<i>+0.0%</i>	<i>+6.2%</i>	

models for single-rule evaluation

5.2 Multi-Rule Performance by Complexity

Table 2 breaks down multi-rule accuracy by the number of required rules. The compositional advantage grows with problem length: +0.7% on 2-rule problems, +1.8% on 3-rule problems, and +16.0% on 4-rule problems.

Table 2: Multi-rule performance breakdown by number of required rules.

Problem Type	Monolithic	Compositional	Advantage
2-rule (75 problems)	1.2%	1.9%	+0.7%
3-rule (60 problems)	8.0%	9.8%	+1.8%
4-rule (40 problems)	12.0%	28.0%	+16.0%

The 4-rule setting shows the largest gap: the compositional model more than doubles the monolithic baseline (28.0% vs 12.0%).

5.3 Error Analysis and Failure Modes

We examined incorrect outputs to identify bottlenecks. The monolithic model shows two dominant failure modes: (1) *rule interference*, where learning one rule degrades performance on another, and (2) *sequence confusion*, where the model applies correct rules in an incorrect order.

The compositional model shows more stable per-rule behavior but fails in two main ways: (1) *decoding errors*, where continuous embeddings decode to invalid expressions, and (2) *energy balancing*, where one rule energy dominates others and pulls optimization toward a single-rule solution basin.

Both models produce valid syntax in more than 95% of cases, so representation and decoding quality is not the main driver of errors. The primary challenge is learning energy landscapes that support reliable multi-step optimization.

6 Discussion

6.1 Why Compositional Energy Models Outperform Monolithic Approaches

The results suggest four mechanisms that favor compositional energies in multi-step reasoning.

Modular specialization. Each rule-specific energy trains on focused examples, which encourages clean rule recognition and reduces cross-rule confusion. A monolithic model must represent multiple rules in one set of parameters, which increases interference.

Combinatorial generalization. The compositional model reuses learned rule energies in new combinations at test time. The model can represent multiple subsets of $\mathcal{R}$ through energy summation without training on each sequence, which aligns with the goal of systematic generalization [Lake and Baroni, 2018].

Energy landscape structure. Individual rule energies can form simpler landscapes around valid local transformations. Summation can preserve useful gradients from multiple rules, while a monolithic energy must represent a more complex multimodal landscape that satisfies competing constraints.

Reduced interference. Independent training avoids catastrophic interference across rules. A monolithic objective can shift the energy surface for one rule while training on another, which can destabilize longer chains that rely on all rules remaining available.

6.2 Connection to Constraint Satisfaction and Modular Reasoning

Our approach parallels compositional constraint satisfaction methods [Yang et al., 2023b] that factor complex problems into simpler constraint components. The energy-based formulation offers three practical benefits for symbolic reasoning: (1) gradient-based optimization provides a continuous search procedure, (2) energy summation naturally expresses conjunction of multiple constraints, and (3) annealing controls exploration and exploitation during inference.

6.3 Implications for Neural-Symbolic Reasoning

Rule-level energies combine interpretability with data-driven learning. Classical symbolic systems guarantee compositionality through explicit rules; our approach learns rule preferences from data

while preserving a modular structure that supports rule recombination. The length-dependent performance gap suggests that modular structure becomes more valuable as reasoning chains grow.

6.4 Limitations and Failure Modes

The approach faces two main limitations. First, decoding from continuous embeddings to discrete expressions introduces approximation errors, especially for complex expressions that require exact symbolic structure. Second, the method assumes a predefined rule set, which limits applicability when rules are unknown or evolve.

The multi-rule accuracy remains low in absolute terms (13.2% overall), which points to several bottlenecks: (1) expression representations may not encode fine-grained algebraic structure, (2) optimization in high-dimensional energy landscapes can stall in local minima, and (3) discrete algebraic transitions do not always align with smooth continuous trajectories.

Future work can address these issues by improving representations (for example, graph neural networks over ASTs), adding hybrid discrete-continuous inference operators, and learning dynamic rule weights conditioned on the current state.

6.5 Broader Impact and Future Directions

Energy-based composition provides a general mechanism for modular reasoning beyond algebra. Natural extensions include geometric reasoning with composable spatial constraints, logical inference with modular premises, and program synthesis where components must compose into longer structures.

The modular design also enables runtime constraint injection, which can add domain-specific requirements without retraining a monolithic model. This direction suggests practical tools for controllable reasoning systems and systematic studies of how architectural modularity affects generalization.

7 Conclusion

We introduce compositional energy-based reasoning for symbolic algebra by training rule-level energy functions and composing them at inference time. The approach improves multi-rule accuracy by 6.2 percentage points over a monolithic IRED baseline (13.2% vs 7.1%) while matching single-rule performance.

Our experiments show that the compositional advantage grows with problem length, culminating in more than doubled accuracy on 4-rule chains (28.0% vs 12.0%). The method supports modularity, enables constraint injection, and provides a concrete framework for studying systematic generalization through architectural decomposition.

References

- Honghua Dong, Jiayuan Mao, Tian Lin, Chong Wang, Lihong Li, and Denny Zhou. Neural logic machines. In *International Conference on Learning Representations*, 2019.
- Yilun Du, Jiayuan Li, Igor Mordatch, and Joshua B Tenenbaum. Iterative reasoning through energy diffusion. In *Proceedings of the 41st International Conference on Machine Learning*, pages 12186–12220. PMLR, 2024.

- Richard Evans and Edward Grefenstette. Learning explanatory rules from noisy data. In *Journal of Artificial Intelligence Research*, volume 61, pages 1–64, 2018.
- Brenden M Lake and Marco Baroni. Generalization without systematicity: On the compositional skills of sequence-to-sequence recurrent networks. *Proceedings of the 35th International Conference on Machine Learning*, 80:2873–2882, 2018.
- Guillaume Lample and François Charton. Deep learning for symbolic mathematics. *arXiv preprint arXiv:1912.01412*, 2019.
- Yann LeCun, Sumit Chopra, Raia Hadsell, Marc’Aurelio Ranzato, and Fu Jie Huang. A tutorial on energy-based learning. *Predicting structured data*, 1(0), 2006.
- Pasquale Minervini, Matko Bosnjak, Tim Rocktäschel, Sebastian Riedel, and Edward Grefenstette. Differentiable reasoning on large knowledge bases and natural language. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018.
- David Saxton, Edward Grefenstette, Felix Hill, and Pushmeet Kohli. Analysing mathematical reasoning abilities of neural models. *arXiv preprint arXiv:1904.01557*, 2019.
- Zhaoyu Yang, Dileep George, Sergey Levine, and Pieter Abbeel. Neural constraint satisfaction: Hierarchical abstraction for combinatorial generalization in robot learning. *arXiv preprint arXiv:2305.19582*, 2023a.
- Zhaoyu Yang, Rao Kashyap, Dileep George, Pieter Abbeel, and Sergey Levine. Compositional diffusion-based continuous constraint solvers. In *Conference on Robot Learning*, pages 1249–1259. PMLR, 2023b.

A Implementation Details

Our implementation builds on IRED with three modifications: (1) separate training datasets and models for each rule, (2) energy composition by weighted summation, and (3) inference under the composed energy. Unless otherwise specified, we use IRED-style defaults for optimization and annealing.

B Additional Results

B.1 Detailed Performance Analysis

We observed several qualitative differences between compositional and monolithic behavior:

Error type distribution. Monolithic models show more rule selection errors (wrong rule preference), while compositional models show more decoding errors (continuous-to-discrete conversion), suggesting different bottlenecks.

Convergence behavior. In our runs, the compositional model typically reaches stable low-energy states in fewer inference steps than the monolithic model, and its energy trajectory is less erratic.

Rule activation patterns. Individual rule energies often exhibit sequential activation patterns consistent with human solution strategies, while monolithic energy gradients are harder to interpret.

B.2 Computational Efficiency

Training the compositional approach trains four models instead of one, which increases total training compute. Inference time remains comparable because energy terms can be computed in parallel, and the modular design enables targeted fine-tuning of one rule model without retraining the full system.